คู่มือ manual-kit

คู่มือการพัฒนา

เครื่องมือทำคู่มือการใช้งานระบบแบบทางการด้วย Claude Code



โครงการ	เลขที่เอกสาร	เวอร์ชัน
manual-kit	**(รอระบุ)**	**0.1**
วันที่	ระดับชั้นความลับ	จัดทำโดย
2026-10-07	**(รอระบุ)**	**ผู้ร่วมพัฒนา manual-kit (ฉบับร่างเขียนโดย Claude Code ด้วย manual-kit)**

ข้อมูลเอกสาร

ชื่อเอกสาร	คู่มือ manual-kit — คู่มือการพัฒนา
ลูกค้า	ผู้ใช้ manual-kit
โครงการ	manual-kit
เลขที่เอกสาร	(รอระบุ)
เวอร์ชัน	0.1
วันที่	2026-10-07
ระดับชั้นความลับ	(รอระบุ)
จัดทำโดย	ผู้ร่วมพัฒนา manual-kit (ฉบับร่างเขียนโดย Claude Code ด้วย manual-kit)

ประวัติการแก้ไข

เวอร์ชัน	วันที่	ผู้แก้ไข	รายละเอียด
0.1	2026-10-07	Claude Code (ร่าง)	ฉบับร่างแรก: คู่มือการใช้งานและคู่มือการพัฒนา เขียนด้วย manual-kit เอง

การอนุมัติเอกสาร

ชื่อ	ตำแหน่ง	ลายมือชื่อ	วันที่
(รอระบุ)	หัวหน้าทีมพัฒนา		

สารบัญ

บทนำ 8

1 สถาปัตยกรรม 9

1.1 สองฝั่ง: เครื่องมือกับ skill 9

1.2 สายงาน 9

1.3 แกนกลางที่ทุกคำสั่งใช้ร่วม 10

1.4 สองแบบของการแจกจ่าย 10

1.5 หน้าที่เกี่ยวข้อง 11

2 โครงสร้าง repo 12

2.1 ระดับบนสุด 12

2.2 ภายใน `plugins/manual-kit/` 12

2.3 โฟลเดอร์ข้อมูลของ plugin 13

2.4 หน้าที่เกี่ยวข้อง 13

3 ตั้งเครื่องสำหรับพัฒนา 14

3.1 ก่อนเริ่ม 14

3.2 ขั้นตอน 14

3.3 ผลลัพธ์ที่ได้ 15

3.4 ปัญหาที่พบบ่อย 15

3.5 หน้าที่เกี่ยวข้อง 15

4 เพิ่ม adapter 16

4.1 ก่อนเริ่ม 16

4.2 ขั้นตอน 16

4.3 ผลลัพธ์ที่ได้ 17

4.4 ปัญหาที่พบบ่อย 17

4.5 หน้าที่เกี่ยวข้อง 18

5 เพิ่มกฎตรวจ 19

5.1 ก่อนเริ่ม 19

5.2 ขั้นตอน 19

5.3 ผลลัพธ์ที่ได้ 20

5.4 ปัญหาที่พบบ่อย 20

5.5 หน้าที่เกี่ยวข้อง 20

6 เพิ่ม action ถ่ายภาพ 21

6.1 ก่อนเริ่ม 21

6.2 ขั้นตอน 21
6.3 ผลลัพธ์ที่ได้ 22
6.4 ปัญหาที่พบบ่อย 22
6.5 หน้าที่เกี่ยวข้อง 22
7 เพิ่มผลลัพธ์หรือช่องทางเผยแพร่ 23
7.1 ก่อนเริ่ม 23
7.2 ขั้นตอน 23
7.3 ผลลัพธ์ที่ได้ 24
7.4 ปัญหาที่พบบ่อย 24
7.5 หน้าที่เกี่ยวข้อง 24
8 ธีม แม่แบบลูกค้า และกฎ DOCX ภาษาไทย 25
8.1 PDF 25
8.2 เว็บคู่มือ (แถบบนสีแบรนด์ เมนูซ้าย "ในหน้านี้" ค้นหา ⌘K) 25
8.3 DOCX 25
8.4 หน้าที่เกี่ยวข้อง 26
9 แก้และทดสอบ skill 27
9.1 ก่อนเริ่ม 27
9.2 ขั้นตอน 27
9.3 ผลลัพธ์ที่ได้ 27
9.4 ปัญหาที่พบบ่อย 27
9.5 หน้าที่เกี่ยวข้อง 28
10 กลยุทธ์การทดสอบ 29
10.1 รันอย่างไร 29
10.2 ส่วนประกอบ 29
10.3 สิ่งที่ตรวจในแต่ละแบบ 29
10.4 หลักการเพิ่มเทสต์ 30
10.5 หน้าที่เกี่ยวข้อง 30
11 ปล่อยเวอร์ชันและกติกาการทำงาน 31
11.1 ก่อนเริ่ม 31
11.2 ขั้นตอน 31
11.3 ผลลัพธ์ที่ได้ 31
11.4 กติกาการทำงานใน repo นี้ 32
11.5 ปัญหาที่พบบ่อย 32
11.6 หน้าที่เกี่ยวข้อง 32
12 บทเรียนที่ได้ 33

12.1 Microsoft Word ค้างระหว่างทำงานอัตโนมัติ 33
12.2 ฟอนต์ไทยใน DOCX ถูกแทนด้วยฟอนต์อื่น 33
12.3 ข้อความถูก escape สองครั้ง 33
12.4 CSS ถูกบล็อกระหว่างถ่ายภาพ 33
12.5 bash 3.2 บน macOS 34
12.6 การหาโมดูลแบบ ESM ใน bundle 34
12.7 คำบรรยายวิดีโอขึ้นสองชั้น 34
12.8 บทเรียนจากการทำคู่มือเล่มนี้ 34
12.9 หน้าที่เกี่ยวข้อง 34

ภาคผนวก 35

สารบัญภาพ

ภาพที่ 3-1 pnpm verify ผ่านทั้งหมด (ผลจริง ตัดเส้นทางโฟลเดอร์ให้สั้น) 14

บทนำ

วัตถุประสงค์

อธิบายว่า manual-kit คืออะไร ใช้ทำคู่มือการใช้งานระบบแบบทางการให้ลูกค้าอย่างไรตั้งแต่ติดตั้งจนส่งมอบและดูแลต่อ (คู่มือการใช้งาน) และอธิบายโครงสร้างภายในกับวิธีต่อยอดเครื่องมือสำหรับนักพัฒนา (คู่มือการพัฒนา)

ขอบเขต

ครอบคลุม plugin สำหรับ Claude Code (คำสั่ง /manual และ skill) คำสั่ง manual-kit ทุกคำสั่ง ผลลัพธ์ PDF, Word, เว็บคู่มือ และ Outline wiki งาน CI บน GitLab และการพัฒนา repo manual-kit ไม่ครอบคลุมวิธีใช้ระบบของลูกค้าแต่ละโครงการ

กลุ่มผู้ใช้

คู่มือการใช้งาน: เพื่อนร่วมทีมที่ต้องทำคู่มือส่งลูกค้า หัวหน้าทีม และผู้จัดการโครงการ (ไม่ต้องเขียนโค้ดเป็น) คู่มือการพัฒนา: นักพัฒนาที่ดูแลหรือเพิ่มความสามารถให้ manual-kit (เขียน TypeScript และใช้ Git เป็น)

คำศัพท์

คำศัพท์	ความหมาย
Claude Code	โปรแกรมผู้ช่วยเขียนโค้ดของ Anthropic ที่ทำงานใน terminal เราสั่งงานด้วยข้อความหรือคำสั่งที่ขึ้นต้นด้วย /
plugin	ชุดส่วนเสริมของ Claude Code ที่ติดตั้งเพิ่มได้ manual-kit มาเป็น plugin ที่มีคำสั่ง /manual, skill และเครื่องมือ manual-kit
marketplace	ที่เก็บรายการ plugin ให้ Claude Code ติดตั้ง repo manual-kit เป็น marketplace ชื่อ manual-kit
skill	เอกสารวิธีทำงานที่ Claude อ่านแล้วทำตาม manual-kit มีสองตัว: manual (ขั้นตอนตั้งแต่ต้นจนจบ) และ building-product-manuals (กติกาการเขียนคู่มือ)
สเปกคู่มือ	ไฟล์ manual-spec.md ที่สรุประบบ ผู้ใช้ ความสามารถ ขั้นตอนการทำงาน และแผนหน้าคู่มือ ทุกข้ออ้างไฟล์ต้นทาง
การรับรอง (sign)	การบันทึกว่าคนอ่านหน้านั้นแล้วและเห็นว่าตรงกับระบบ ทำโดยคนเท่านั้น
(รอระบุ)	เครื่องหมายแทนข้อมูลที่ยังไม่รู้ เช่น เลขที่เอกสารหรือผู้อนุมัติ ต้องกรอกก่อนส่งลูกค้า

1 สถาปัตยกรรม

manual-kit แบ่งงานเป็นสองฝั่งที่ไม่ก้าวก่ายกัน การเข้าใจเส้นแบ่งนี้คือกุญแจของการแก้ repo: งานไหนต้อง **ให้ผลเหมือนเดิมทุกครั้ง** อยู่ในเครื่องมือ งานไหนต้อง **ใช้วิจารณญาณ** อยู่ใน skill

1.1 สองฝั่ง: เครื่องมือกับ skill

	เครื่องมือ `manual-kit` (src/)	**skill (plugins/manual-kit/skills/)**
ทำอะไร	สแกนโค้ด ตรวจ ถ่ายภาพ ทำวิดีโอ ออกไฟล์ เผยแพร่	อ่านโครงการ เขียนสเปก เลือกโครงเล่ม เขียนเนื้อหา ดูภาพแล้วตัดสินว่าถูก
ลักษณะ	deterministic: รันซ้ำได้ผลเดิม ไม่มี timestamp ในผลที่ commit ไม่เรียก AI ไม่ใช้ API key	ทำตามกติกาใน SKILL.md ด้วยวิจารณญาณของ Claude
ทดสอบอย่างไร	vitest รัน CLI จริงกับโครงการตัวอย่าง	เทียบผลการทำงานของ Claude แบบไม่มีและมี skill (แก้และทดสอบ skill)
ทำไม	ใช้ใน CI ได้ ผล diff ได้ เชื่อถือได้	ความเข้าใจผู้อ่านและภาษาที่อ่านง่ายเขียนเป็นสคริปต์ไม่ได้

กฎข้อเดียวที่ห้ามข้าม: **ไม่มีการเรียก AI ในเครื่องมือ** ถ้างานไหนต้องใช้วิจารณญาณ ให้เครื่องมือรวบรวมหลักฐานหรือตรวจผล แล้วให้ skill เป็นคนตัดสิน เช่น `discover` เก็บหลักฐานแต่ไม่เขียนสเปก `check` ชี้หน้าที่ต้องแก้แต่ไม่แก้

1.2 สายงาน

ลำดับ	ขั้น	ใครทำ	คำสั่ง	ผลลัพธ์
1	discover	เครื่องมือ	`manual-kit discover`	`discovery.json` + `discovery.md` (เรียงลำดับ ไม่มีเวลา จึง diff ได้)
2	spec	skill		`manual-spec.md` ทุกข้ออ้างไฟล์
3	pages	skill (+ `init`, `inventory`, `new-page`)		`pages/<lang>/<book>/NN-slug.md`, `coverage.yaml`
4	capture	เครื่องมือ (spec จาก skill)	`manual-kit capture`	`media/<lang>/*.png`, `manifest.json` (hash ของ spec ต่อภาพ)
5	encode	เครื่องมือ	`manual-kit encode`	MP4 + ภาพปก + WebVTT
6	build	เครื่องมือ	`manual-kit build`	PDF, DOCX, เว็บ ใน `dist/`

7	check	เครื่องมือ (skill อ่านผลแล้วแก้)	`manual-kit check`	รายการ error/warning, exit 1 เมื่อผิด
8	publish	เครื่องมือ	`manual-kit publish outline`	เอกสารใน Outline + `outline.json`

ลำดับจริงวนกลับได้: check ชี้ปัญหา → แก้หน้าหรือ shots.yaml → capture --changed → build ใหม่ และ **คน** รับรอง (`sign`) ระหว่าง check กับการส่งมอบ เครื่องมือมีคำสั่งรับรองให้คนเรียก แต่ skill ห้ามเรียก

1.3 แกนกลางที่ทุกคำสั่งใช้ร่วม

- **Project** (`src/project.ts`): `openProject(manualDir)` อ่าน config (ตรวจด้วย zod ที่ขอบ `src/config.ts`) หน้าทั้งหมด (`src/pages.ts`) และสร้าง adapter ทุกคำสั่งหลัง `init` เริ่มจากตรงนี้ ภายในเชื่อ type ได้
- **Adapter** (`src/adapters/types.ts`): อินเทอร์เฟซเดียวที่อ่านโค้ดของโครงการ ไม่รันโค้ดนั้น

```
export interface Adapter {
  /** Routes, roles, permissions, sections and flags, sorted by id, no duplicates.
*/
  items(): InventoryItem[]
  /** UI string key -> text for one language (empty map when the language is
unknown). */
  uiStrings(lang: string): Map<string, string>
  /** A scalar value from the project (`ns.path.to.value`), or undefined. */
  data(key: string): string | undefined
}
```

- **Tokens** (`src/pages.ts` `mapTokens`): ตัวสแกนของ check และตัวแทนค่าของ build ใช้ฟังก์ชันเดียวกัน token ที่ check เห็นจึงเป็น token เดียวกับที่ build แทนค่า (ข้ามเฉพาะในโค้ดบล็อกแบบรั้ว)
- **Render** (`src/render.ts`): หน้า Markdown → ผลลัพธ์ตาม target `html` (PDF), `web`, `pandoc` (DOCX), `outline` ผลลัพธ์ใหม่จึงมักเริ่มจากการเพิ่ม target (เพิ่มผลลัพธ์หรือช่องทางเผยแพร่)
- **Lock** (`src/lock.ts`): `manual.lock.json` เก็บ hash ของไฟล์หน้าและ `sources` ตอนรับรอง check เทียบกับปัจจุบัน

1.4 สองแบบของการแจกจ่าย

1. **repo** (นักพัฒนา): `node dist/cli.js` หลัง `pnpm build` หรือรัน `src/cli.ts` ตรงด้วย Node 22.18 ขึ้นไป (เทสต์ทำแบบนี้)
2. **plugin** (ทีม): `plugins/manual-kit/dist/manual-kit.mjs` ไฟล์เดียวที่ bundle แล้ว แพ็กเกจหนัก (playwright, pagedjs, typescript) ไม่ bundle แต่ติดตั้งลงโฟลเดอร์ข้อมูลของ plugin ตอนเปิดครั้งแรก (โครงสร้าง repo, ปล่อยเวอร์ชันและกติกาการทำงาน)

1.5 หน้าที่เกี่ยวข้อง

- โครงสร้าง repo
- ตั้งเครื่องสำหรับพัฒนา

2 โครงสร้าง repo

repo เดียวทำสามหน้าที่: เป็นซอร์สของเครื่องมือ เป็น marketplace ของ Claude Code และเป็นตัว plugin ที่ทีมติดตั้ง ตารางด้านล่างบอกว่าโฟลเดอร์ไหนแก้มือ โฟลเดอร์ไหนสร้างจากสคริปต์ (แก้มือแล้วจะถูกเขียนทับ)

2.1 ระดับบนสุด

ที่อยู่	คืออะไร	แก้มือ?
`src/`	ซอร์ส TypeScript ของ CLI ทั้งหมด (ESM, Node 22.18 ขึ้นไปรัน `.ts` ได้ตรง)	ใช่
`assets/`	ฟอนต์ Sarabun (OFL), `pdf.css`, ธีมเว็บ `web/site.css` + `site.js`, `libreoffice/finalize.py`	ใช่
`templates/pages/`	แม่แบบหน้า concept / reference ต่อภาษา (how-to มาจาก `src/how-to.ts`)	ใช่
`tests/`	เทสต์ vitest, `fixtures/` โครงการตัวอย่าง, `helpers/`	ใช่
`scripts/bundle-plugin.mjs`	สร้างสำเนาของ plugin จากซอร์ส	ใช่
`.claude-plugin/marketplace.json`	ประกาศ marketplace `manual-kit` ที่มี plugin `manual-kit`	ใช่
`plugins/manual-kit/`	ตัว plugin (ดูตารางถัดไป)	บางส่วน
`docs/`	`reference.md` (คู่มืออ้างอิงคำสั่ง), `spec.md` (สเปกการออกแบบ), `examples/`, `images/`, `media/` (วิดีโอแนะนำ), `manual/` (คู่มือเล่มนี้), `engineering-principles.md`	ใช่
`dist/`	ผลของ `pnpm build` (tsc) สำหรับรันจาก repo	ไม่ (ignore)

2.2 ภายใน `plugins/manual-kit/`

ที่อยู่	คืออะไร	มาจาก
`dist/manual-kit.mjs`	CLI ไฟล์เดียว bundle ด้วย esbuild (target node22)	`pnpm bundle` **commit ด้วย**
`assets/`, `templates/`	สำเนาจากระดับบน	`pnpm bundle`
`runtime/package.json` (+ lock)	แพ็กเกจที่ไม่ bundle: playwright, pagedjs, typescript รุ่นที่ตรงกับ `node_modules`	`pnpm bundle`

`bin/manual-kit`	ตัวเรียก bash: `setup`, `doctor`, แล้วรัน bundle	แก้มือ
`scripts/env.sh`, `scripts/setup.sh`	หาโฟลเดอร์ข้อมูล (`MANUAL_KIT_HOME`), ตรวจ Node, ติดตั้ง runtime และ Chromium	แก้มือ
`hooks/hooks.json`	SessionStart: รัน `setup.sh --background` ตอนเปิด Claude Code	แก้มือ
`skills/manual/SKILL.md`	คำสั่ง `/manual` (ขั้นตอน)	แก้มือ
`skills/building-product-manuals/`	`SKILL.md` (กติกา), `spec-template.md`, `reference.md`	`reference.md` คัดลอกจาก `docs/reference.md` โดย `pnpm bundle`

ทำไม bundle ต้อง commit: ทีมติดตั้ง plugin ด้วยการ clone repo ไม่มีขั้น build บนเครื่องของเพื่อนร่วมทีม ไฟล์ที่รันได้จึงต้องอยู่ใน git เทสต์ `tests/plugin-bundle.test.ts` สร้าง bundle ใหม่แล้วเทียบกับที่ commit ไว้ ถ้าแก้ `src/` แล้วลืม `pnpm bundle` เทสต์จะไม่ผ่าน

2.3 โฟลเดอร์ข้อมูลของ plugin

แพ็กเกจ runtime และ Chromium อยู่นอก plugin ที่ `~/.claude/plugins/data/manual-kit-manual-kit` (หรือ `MANUAL_KIT_HOME`) จึงไม่ต้องติดตั้งใหม่ทุกครั้งที่อัปเดต plugin `src/deps.ts` โหลดแพ็กเกจเหล่านี้จากที่นั่นเมื่อใช้ครั้งแรก คำสั่งที่ไม่ต้องใช้ (เช่น `--version`, `check`) จึงรันได้แม้ยังไม่ติดตั้ง

2.4 หน้าที่เกี่ยวข้อง

- สถาปัตยกรรม
- ตั้งเครื่องสำหรับพัฒนา
- ปล่อยเวอร์ชันและกติกาการทำงาน

3 ตั้งเครื่องสำหรับพัฒนา

หน้านี้พาตั้งเครื่องให้รันเทสต์ทั้งหมดผ่าน และลอง plugin ที่แก้แล้วใน Claude Code แบบแยกจาก config จริง

3.1 ก่อนเริ่ม

- Node >=22.17 คือขั้นต่ำของ CLI และ plugin (ใช้ `fs.globSync` ที่เสถียรตั้งแต่ 22.17) แต่การพัฒนา repo รันไฟล์ `.ts` ตรง จึงต้อง Node 22.18 ขึ้นไป (แนะนำ Node 24) เลขขั้นต่ำต้องตรงกันใน package.json, `scripts/env.sh`, README และ docs/reference.md (เทสต์ `tests/node-version.test.ts` ตรวจให้)
- pnpm, git, สิทธิ์ push repo manual-kit (หรือ fork ของคุณ)
- สำหรับเทสต์บางชุด: pandoc (DOCX), ffmpeg (คลิป), poppler `pdftotext` (อ่านข้อความใน PDF)

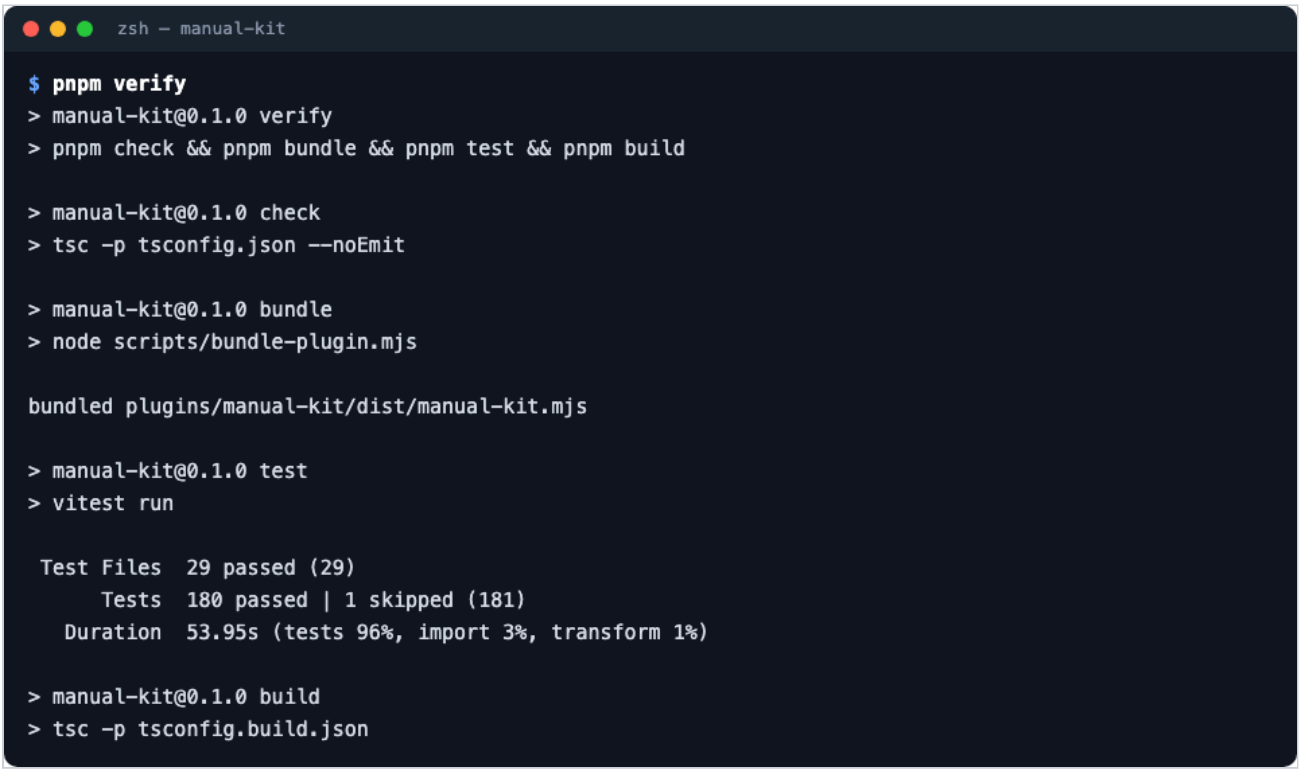
3.2 ขั้นตอน

1. ติดตั้งแพ็กเกจและ Chromium ของ Playwright:

```
pnpm install && pnpm exec playwright install chromium
```

2. ตรวจทั้งหมดก่อนเริ่มแก้ เพื่อรู้ว่าเริ่มจากสถานะที่ผ่าน:

```
pnpm verify   # = pnpm check && pnpm bundle && pnpm test && pnpm build
```



ภาพที่ 3-1 pnpm verify ผ่านทั้งหมด (ผลจริง ตัดเส้นทางโฟลเดอร์ให้สั้น)

3. รัน CLI จาก repo: `node dist/cli.js <คำสั่ง>` หลัง `pnpm build` หรือ `node src/cli.ts <คำสั่ง>` ตรงจากซอร์ส ใช้ **--manual** ชี้โฟลเดอร์คู่มือ เช่น `node dist/cli.js -m docs/manual check --allow-unsigned`
4. หลังแก้ `src/` ทุกครั้ง รัน `pnpm bundle` แล้ว commit `plugins/manual-kit/dist/manual-kit.mjs` ไปด้วย
5. ทดสอบ plugin ใน Claude Code **ด้วยโฟลเดอร์ config ชั่วคราวเสมอ** เพื่อไม่ให้ไปเปลี่ยน plugin หรือการตั้งค่าที่ใช้ทำงานจริง:

```
export CLAUDE_CONFIG_DIR=/tmp/mk-plugin-test     # โฟลเดอร์ทิ้งได้
claude plugin marketplace add /path/to/manual-kit  # ชี้ที่ repo ในเครื่อง ไม่ต้อง push ก่อน
claude plugin install manual-kit@manual-kit
claude                                            # แล้วลอง /manual ในโครงการทดสอบ
```

เมื่อเสร็จ ลบโฟลเดอร์นั้นทิ้ง ห้ามใช้ `~/.claude` ของตัวเองหรือของคนอื่นเป็นที่ทดสอบ

3.3 ผลลัพธ์ที่ได้

`pnpm verify` ผ่าน (ทุกไฟล์เทสต์ผ่าน) และ plugin ที่แก้แล้วทำงานใน Claude Code ชุดทดสอบ

3.4 ปัญหาที่พบบ่อย

อาการ	สาเหตุ	วิธีแก้
เทสต์ `is committed as built from the current source (run: pnpm bundle)` ไม่ผ่าน	แก้ `src/` แต่ bundle เก่า	`pnpm bundle` แล้ว commit
เทสต์ DOCX หรือคลิปถูกข้าม/ไม่ผ่าน	ไม่มี pandoc หรือ ffmpeg	`brew install pandoc ffmpeg`
`unknown command 'doctor'`	`doctor` และ `setup` อยู่ในตัวเรียก plugin (`bin/manual-kit`) ไม่อยู่ใน CLI	รัน `plugins/manual-kit/bin/manual-kit doctor`
`bin/manual-kit` พังบน macOS	bash 3.2 ของ macOS ไม่รับ `case` ใน `$(...)`	ดู บทเรียนที่ได้

3.5 หน้าที่เกี่ยวข้อง

- กลยุทธ์การทดสอบ
- ปล่อยเวอร์ชันและกติกาการทำงาน

4 เพิ่ม adapter

adapter คือส่วนที่อ่านโค้ดของโครงการแล้วตอบสามคำถาม: มีหน้าจอ/บทบาท/flag อะไรที่ต้องมีหน้าคู่มือ (`items`) ป้ายบนหน้าจอแต่ละภาษาคืออะไร (`uiStrings`) และค่าข้อมูลจากไฟล์ (`data`) เพิ่ม adapter เมื่อเฟรมเวิร์กใหม่มีโครงที่ regex ของ adapter `generic` อธิบายไม่ได้ดี เช่น router แบบไฟล์ ก่อนเขียนใหม่ ลองตั้ง `generic` ใน config ก่อนเสมอ (หลักการข้อ 1 ขี้เกียจอย่างมีวินัย)

4.1 ก่อนเริ่ม

- อ่าน สถาปัตยกรรม หัวข้อ Adapter
- มีโครงการตัวอย่างขนาดเล็กของเฟรมเวิร์กนั้น (ไม่กี่ไฟล์) สำหรับเป็น fixture

4.2 ขั้นตอน

1. **สร้าง fixture** ใน `tests/fixtures/<ชื่อ>/` ที่มีทุกกรณีที่ต้องรองรับ เช่น `tests/fixtures/next-app` มี route group `(marketing)`, parallel slot `@modal`, โฟลเดอร์ private `_private`, `api/`, dynamic `[id]` และ `[...slug]`

2. **เขียนเทสต์ก่อน** (TDD) ใน `tests/frameworks.test.ts` เรียก adapter แบบที่ผู้ใช้เรียกผ่าน config:

```
const adapter = (cfg: unknown, root: string) =>
createAdapter(AdapterConfig.parse(cfg), root)

it('App Router: page files become routes; groups stripped, dynamic kept,
parallel/intercepting/private ignored', () => {
  const a = adapter({ type: 'nextjs' }, fx('next-app'))
  expect(ids(a.items())).toEqual(['route:/', 'route:/about', 'route:/blog',
'route:/dashboard/[id]', 'route:/docs/[...slug]'])
  expect(a.items().find((i) => i.id === 'route:/about')).toMatchObject({
source: 'app/(marketing)/about/page.tsx' })
})
```

3. **เพิ่ม schema** ใน `src/config.ts`: วัตถุ zod ที่มี `type: z.literal('<ชื่อ>')` ใส่ `...Common` (roles, flags, sections, data) แล้วเพิ่มเข้า `AdapterConfig` (discriminated union) และตัวเลือก `--adapter` ของ `init` ใน `src/cli.ts`

4. **เขียน adapter** อ่านไฟล์เป็นข้อความหรือ AST เท่านั้น **ห้ามรันโค้ดของโครงการ** ตัวอย่างการหา route จาก App Router:

```
function nextRoutes(root: string): InventoryItem[] {
  const list = new ItemList()
  const app = firstDir(root, ['app', 'src/app'])
  if (app) {
    for (const rel of files(root, [`${app}/**/page.{tsx,jsx,ts,js,mdx}`])) {
      const segs = rel.slice(app.length + 1).split('/').slice(0, -1)
      // private folders (_x) are not routable; slots and intercepting routes
render inside another page
      if (segs.some((s) => s.startsWith('_') || isSlotOrIntercept(s)))
continue
      list.add('route', toRoute(segs), { source: rel })
    }
  }
  // ... Pages Router
  return list.list()
}
```

`ItemList` เรียงลำดับและตัดซ้ำให้ (ผลต้องเหมือนเดิมทุกครั้ง) ใส่ `source` ทุกรายการเพื่อให้ discovery และ coverage บอกที่มาได้ ป้ายบนหน้าจอ: ถ้ามีไฟล์ i18n ใช้ไฟล์นั้น ถ้าไม่มี nextjs ใช้ TypeScript parser อ่านข้อความใน JSX (`src/adapters/literals.ts`) โดยให้ข้อความเป็น key ของตัวเอง หน้าคู่มือจึงอ้างแบบมีเครื่องหมายคำพูดได้

5. **ต่อเข้ากับ `baseAdapter`** ใน `src/adapters/index.ts` (เพิ่ม case ใหม่ TypeScript จะเตือนถ้าลืมเพราะ union ครบทุกชนิด) `sections` ทำงานให้ทุก adapter อยู่แล้วผ่าน `createAdapter`
6. ถ้าเฟรมเวิร์กมีสัญญาณให้เดา ให้ `discover` เสนอ adapter นี้ (ดู `src/discover.ts`) และเพิ่มแถวในตาราง Adapters ของ `docs/reference.md`
7. `pnpm verify` แล้ว commit เทสต์ + โค้ด + bundle + เอกสาร ในเรื่องเดียว

4.3 ผลลัพธ์ที่ได้

`init --adapter <ชื่อ>` ใช้ได้ `inventory` ได้หน้าจอของโครงการ และตัวอ้างอิงป้าย (ui) ในหน้าคู่มืออ้างป้ายบนหน้าจอจริงได้ เมื่อมีคนเปลี่ยนชื่อปุ่มในโค้ด check ของโครงการนั้นจะไม่ผ่าน

4.4 ปัญหาที่พบบ่อย

อาการ	สาเหตุ	วิธีแก้
ผล `items()` สลับลำดับระหว่างเครื่อง	วนไฟล์ตามลำดับของระบบไฟล์	ผ่าน `ItemList` เสมอ
ป้ายที่มีตัวแปรปน (เช่น จำนวนรายการ) หายไป	ข้อความที่ติดกับ expression ไม่ใช่ป้ายคงที่	ตั้งใจ: คู่มือควรอ้างป้ายคงที่

โหลด typescript ไม่ได้ใน plugin	typescript ไม่ได้ bundle	โหลดผ่าน `lazyDep` / `dep` จาก `src/deps.ts`

4.5 หน้าที่เกี่ยวข้อง

- เพิ่มกฎตรวจ
- กลยุทธ์การทดสอบ

5 เพิ่มกฎตรวจ

check คือหัวใจที่ทำให้คู่มือตามระบบ ทุกครั้งที่ทีมเจอความผิดพลาดซ้ำในคู่มือ (คำผิดแบบเดิม หัวข้อขาด ภาพเก่า) ให้คิดก่อนว่าเปลี่ยนเป็นกฎได้ไหม แทนการเขียนเตือนเพิ่มใน skill (หลักการ #23) `src/check.ts` เป็นฟังก์ชันบริสุทธิ์ อ่านไฟล์ของโครงการแล้วคืนรายการ finding ไม่เขียนอะไรเลย จึงทดสอบง่ายและใช้ใน CI ได้

5.1 ก่อนเริ่ม

- ตัดสินว่าเป็น **error** (ผิดแน่ ต้องแก้ก่อนส่ง เช่น ป้ายหายจากโค้ด) หรือ **warning** (ควรแก้ แต่มีเหตุผลที่จะยอมได้ เช่น หัวข้อ how-to)
- ถ้ากฎต้องใช้ข้อมูลร่วมกับคำสั่งอื่น ให้วางข้อมูลนั้นไว้ที่เดียว (ตัวอย่าง: หัวข้อ how-to อยู่ใน `src/how-to.ts` ใช้ทั้ง `new-page` ที่เขียนหัวข้อ และ check ที่ตรวจหัวข้อ)

5.2 ขั้นตอน

1. **เขียนเทสต์ก่อน** ใน `tests/check.test.ts`: คัดลอก fixture ที่ผ่านทุกกฎ ทำให้ผิดหนึ่งจุด แล้วรัน CLI จริงด้วย `--json`:

```
it('structure: a how-to page missing a required part (or with parts out of
order) is a warning naming them', async () => {
  const root = await cleanFixture()
  edit(root, EN1, (s) => s.replace('## Common problems', '## Troubles'))
  edit(root, TH1, (s) => s.replace(/## ผลลัพธ์ที่ได้[\s\S]*?(?=## ปัญหาที่พบบ่อย)/, ''))
  await signAll(root)
  const r = check(root)
  expect(r.code).toBe(0)
  const s = r.findings.filter((f) => f.rule === 'structure')
  expect(s[0]!.message).toMatch(/missing "Common problems"/)
})
```

2. **เพิ่มชื่อกฎ** ใน union `Rule` ของ `src/check.ts` (ชื่อนี้ขึ้นในคอลัมน์ที่สองของผล check และใน `--json`)

3. **เพิ่มการตรวจ** ใน `runCheck` ด้วย `err(rule, message, file, line?)` หรือ `warn(rule, message, file)` ข้อความต้องบอกทั้ง **อะไรผิด** และ **ทำอย่างไรต่อ** เพราะคนที่อ่านคือ Claude หรือเพื่อนร่วมทีมใน CI:

```
if (p.meta.kind === 'how-to' && howToLangs(p.lang)) {
  const problems = howToProblems(p.body, p.lang)
  if (problems.length)
    warn('structure', `how-to page ${p.id}: ${problems.join(', ')} (a how-to
has ${HOW_TO_PARTS.map((h) => `"${h[p.lang as 'th' | 'en']}"`).join(', ')} as
## headings)`, p.rel)
}
```

4. ถ้ากฎตั้งค่าได้ ให้เพิ่มช่องใน `ConfigSchema` (`src/config.ts`) พร้อมค่าเริ่มต้น เช่น `wording.forbidden` กฎของโครงการหนึ่ง (เช่น "ใช้คำว่า กด") ควรเป็นค่าใน config ไม่ใช่โค้ดใหม่
5. อัปเดตหัวข้อ "What check verifies" ใน `docs/reference.md` และ CHANGELOG แล้ว `pnpm verify`

5.3 ผลลัพธ์ที่ได้

check รายงานกฎใหม่ในทุกโครงการ ทั้งบนเครื่องและใน CI โดยไม่มีใครต้องจำ ตัวอย่างจริง: ระหว่างเขียนคู่มือเล่มนี้ กฎ `wording` จับคำโอ้อวดที่หลุดมาในประโยคธรรมดาได้สามที่ และกฎ `ui` จับป้ายของคำสั่งที่ adapter ยังไม่รู้จักได้

5.4 ปัญหาที่พบบ่อย

อาการ	สาเหตุ	วิธีแก้
เทสต์ของกฎอื่นพังตาม	fixture กลางไม่ผ่านกฎใหม่	แก้ fixture ให้ถูกตามกฎ (fixture กลางต้องผ่านทุกกฎ)
กฎตรวจโค้ดในโค้ดบล็อกด้วย	อ่าน `p.body` ตรง	ถ้ากฎเกี่ยวกับ token ใช้ `p.tokens` ซึ่งข้ามโค้ดบล็อกแล้ว
คำต้องห้ามโดนในประโยคที่อธิบายกฎเอง	regex ไม่รู้บริบท	เขียนประโยคใหม่ หรือจำกัดภาษาด้วย `lang`

5.5 หน้าที่เกี่ยวข้อง

- กลยุทธ์การทดสอบ
- เพิ่ม action ถ่ายภาพ

6 เพิ่ม action ถ่ายภาพ

action คือสิ่งที่ capture ทำบนหน้าเว็บก่อนวาดกรอบแดงและถ่ายภาพ เช่น กด กรอก เลือก ชี้เมาส์ ทุก action มีสองที่: schema ใน `src/shots.ts` (ตรวจ shots.yaml ที่ขอบ) และตัวรันใน `src/capture.ts` ฟังก์ชัน `run`

6.1 ก่อนเริ่ม

- action ใหม่ต้องทำซ้ำได้ผลเดิม และไม่ออกนอก allowlist (การเปิดหน้าใหม่ต้องผ่าน `goto` ที่ตรวจ host เสมอ)
- ถ้ารับข้อความจากผู้ใช้ ให้ผ่าน `fill(value, lang)` เพื่อรองรับ `{lang}` และ `{env:NAME}` (รหัสผ่านไม่อยู่ในไฟล์)

6.2 ขั้นตอน

1. **เขียนเทสต์ก่อน** ใน `tests/capture.test.ts`: หน้า HTML เล็ก ๆ ที่เซิร์ฟจาก `tests/helpers/server.ts` บันทึกผลของ action ลงหน้าเอง (เช่น `onmouseenter="this.dataset.hover = '1'"`) แล้วตรวจจากภาพหรือ DOM ว่า action ทำงานก่อนถ่าย

2. **เพิ่ม schema** หนึ่งแถวใน union `Action` (ใช้ `.strict()` เพื่อให้พิมพ์ผิดแล้ว shots.yaml ไม่ผ่าน แทนการถูกข้ามเงียบ ๆ):

```
export const Action = z.union([
  z.object({ click: Selector }).strict(),
  z.object({ fill: Selector, value: z.string() }).strict(),
  /** A key (Playwright name: Enter, Escape, Control+A...), on the focused
element or on `on`. */
  z.object({ press: z.string().min(1), on: Selector.optional() }).strict(),
  z.object({ hover: Selector }).strict(),
  // ...
])
```

3. **เพิ่มตัวรัน** หนึ่งบรรทัดใน `run` ของ `src/capture.ts`:

```
else if ('press' in a) await (a.on ? page.press(a.on, a.press) :
page.keyboard.press(a.press))
else if ('hover' in a) await page.hover(a.hover)
```

4. ถ้า action ใช้ได้เฉพาะบางชนิดของ spec (เช่น `show` ใช้ได้กับหน้าต่าง terminal เท่านั้น) ให้ตรวจใน `loadShots` แล้วโยนข้อความที่บอกวิธีแก้ เช่น `"show" works only in a terminal spec`

5. action เป็นส่วนของ spec hash อยู่แล้ว (`specHash`) การแก้ action ใน shots.yaml จึงทำให้ `capture --changed` ถ่ายภาพนั้นใหม่เอง

6. เพิ่มแถวในตาราง action ของ `docs/reference.md` (หัวข้อ shots.yaml) แล้ว `pnpm verify`

6.3 ผลลัพธ์ที่ได้

ผู้เขียนคู่มือใช้ action ใหม่ใน shots.yaml ได้ทันที ทั้งใน `shots`, `steps`, `clips` และ `logins`

6.4 ปัญหาที่พบบ่อย

อาการ	สาเหตุ	วิธีแก้
ภาพไม่มีสไตล์	CSS หรือฟอนต์จาก CDN ถูกบล็อกเพราะไม่อยู่ใน allowlist	capture ไม่ผ่านพร้อม `stylesheet blocked` ให้เพิ่ม host ใน `capture.assetHosts` (ดู บทเรียนที่ได้)
action กดโดนแถบเครื่องมือ dev ของเฟรมเวิร์ก	overlay ของ Next.js / Vite บังปุ่ม	overlay ที่รู้จักถูกซ่อนตั้งแต่โหลดหน้า (`DEV_OVERLAYS` ใน `src/config.ts`) เพิ่มตัวใหม่ที่นั่น
กรอบแดงอยู่ใต้ modal	ปุ่มอยู่ใน `dialog:modal`	capture วาดกรอบเป็น popover เหนือ top layer ให้แล้ว

6.5 หน้าที่เกี่ยวข้อง

- เพิ่มผลลัพธ์หรือช่องทางเผยแพร่
- กลยุทธ์การทดสอบ

7 เพิ่มผลลัพธ์หรือช่องทางเผยแพร่

ผลลัพธ์ทุกแบบ (PDF, DOCX, เว็บ, Outline) เริ่มจากที่เดียวกัน: `renderBook` ใน `src/render.ts` แปลงหน้า Markdown พร้อม token ให้เป็นข้อความตาม **target** แล้วตัวสร้างแต่ละแบบนำไปใช้ต่อ การเพิ่มช่องทางใหม่จึงมีสามส่วน: target ใน render, ตัวสร้างหรือตัวส่ง, และคำสั่งใน CLI

7.1 ก่อนเริ่ม

- อ่าน API ของปลายทาง และหาวิธีทดสอบโดยไม่ยิงของจริง (Outline ใช้เซิร์ฟเวอร์จำลอง `tests/helpers/outline-mock.ts`)
- ความลับ (API key) อ่านจากตัวแปรสภาพแวดล้อมเท่านั้น ห้ามเขียนลงไฟล์ ห้ามพิมพ์ออกหน้าจอ

7.2 ขั้นตอน

1. **เพิ่ม target** ใน `Target` ของ `src/render.ts` แล้วจัดการจุดที่ต่างกัน เช่น Outline ไม่มีวิดีโอในเอกสาร จึงแสดงภาพปกพร้อมลิงก์ MP4 และ callout กลายเป็น notice ของ Outline:

```
export type Target = 'html' | 'web' | 'pandoc' | 'outline'
// ...
if (target === 'outline') return `\n${figure(f.poster, caption, 'clip',
alt)}\n\n[▶ ${escMd(`${L.play}: ${caption}`)} (MP4)](${f.mp4})\n`
```

2. **วางแผนก่อนส่ง**: ฟังก์ชัน `plan` คืนรายการเอกสาร (เล่มละหนึ่ง หน้าละหนึ่ง) พร้อมไฟล์ที่ต้องแนบ โดยยังไม่ส่งอะไร ทั้ง `--dry-run` และการส่งจริงใช้แผนเดียวกัน สิ่งที่ dry-run แสดงจึงตรงกับสิ่งที่จะเกิดจริง

3. **ทำให้ส่งซ้ำได้ผลเดิม** (หลักการ #14): เก็บ mapping ไว้ในไฟล์ที่ commit ได้ (`manual/outline.json`: รหัสเอกสารต่อหน้า, hash ของข้อความที่ส่งล่าสุด, ไฟล์แนบตาม sha256 ของไฟล์) ส่งครั้งต่อไปจึงแก้เฉพาะที่เปลี่ยน อัปโหลดเฉพาะไฟล์ใหม่ และกันการส่งผิดที่ด้วยการผูกไฟล์ไว้กับ wiki และ collection เดียว

4. **อ่าน key จากสภาพแวดล้อมเท่านั้น** และบอกวิธีแก้เมื่อไม่มี:

```
export const TOKEN_ENV = 'OUTLINE_API_KEY'
// ... throw new UsageError(`set the Outline API key in the environment
variable ${TOKEN_ENV} (it is never read from a file)`)
```

5. **ต่อเข้า CLI** ใน `src/cli.ts` ใต้คำสั่งกลุ่ม `publish` (หรือเพิ่มรูปแบบให้ `build` ถ้าเป็นไฟล์):

```
const publish = program.command('publish').description('publish the built
manual somewhere people read it')
publish
  .command('outline')
  .option('--collection <id>', 'collection id or urlId (default: config
outline.collection)')
  .option('--dry-run', 'print what would be created, updated and uploaded;
send nothing')
  .action(async (opts) => { await publishOutline(await
openProject(manualDir()), opts) })
```

6. เพิ่มค่าใน `ConfigSchema` ถ้ามี (เช่น `outline: { baseUrl, collection, parent }`) เทสต์กับ mock (สร้าง แก้ ไม่ส่งซ้ำ สร้างใหม่เมื่อถูกลบ ไม่มี key) อัปเดต `docs/reference.md` และ CHANGELOG แล้ว `pnpm verify`
7. คำสั่งใหม่เป็นรายการ coverage ของคู่มือเล่มนี้โดยอัตโนมัติ (adapter ของเล่มนี้อ่าน `.command(...)` ใน `src/cli.ts`) check ของ `docs/manual` จะไม่ผ่านจนกว่าจะมีหน้าคู่มือพูดถึงคำสั่งนั้น

7.3 ผลลัพธ์ที่ได้

ช่องทางใหม่ที่มี dry-run ส่งซ้ำได้ปลอดภัย ไม่รั่วความลับ และมีหน้าคู่มือกำกับ

7.4 ปัญหาที่พบบ่อย

อาการ	สาเหตุ	วิธีแก้
ลิงก์ข้ามหน้าเสียบนปลายทาง	ลิงก์ชี้ anchor ภายในไฟล์	เขียนลิงก์ใหม่หลังรู้ที่อยู่เอกสาร (`finalText` แทนลิงก์ด้วย URL ของเอกสาร)
ส่งซ้ำแล้วได้เอกสารซ้ำ	ไม่มี mapping หรือไม่ได้ commit	เก็บ mapping และ commit
ภาพไม่ขึ้น	ส่งที่อยู่ไฟล์ในเครื่อง	อัปโหลดเป็นไฟล์แนบก่อน แล้วแทนที่อยู่ด้วย URL ของไฟล์แนบ

7.5 หน้าที่เกี่ยวข้อง

- ธีม แม่แบบลูกค้า และกฎ DOCX ภาษาไทย
- กลยุทธ์การทดสอบ

8 ธีม แม่แบบลูกค้า และกฎ DOCX ภาษาไทย

หน้าตาของผลลัพธ์แต่ละแบบอยู่คนละที่ หน้านี้รวมไว้ให้รู้ว่าจะแก้ธีมกลาง (ทุกโครงการได้ผล) หรือให้โครงการใส่แม่แบบของตัวเอง (เฉพาะโครงการนั้น) และอธิบายกฎภาษาไทยของ DOCX ที่ห้ามทำหาย

8.1 PDF

ส่วน	ที่อยู่
HTML สำหรับพิมพ์ (ปก ข้อมูลเอกสาร ประวัติ อนุมัติ สารบัญ บท)	`src/build/html.ts`
สไตล์กลาง (A4 หัว-ท้ายกระดาษ "หน้า X จาก Y" เลขบท ภาพ)	`assets/pdf.css`
จัดหน้าด้วย Paged.js ใน Chromium ย่อภาพที่เหลือที่ไม่พอ	`src/build/pdf.ts`
แม่แบบของโครงการ	`templates.pdfCss` ใน config: CSS ต่อท้ายสไตล์กลาง

ฟอนต์เริ่มต้นคือ Sarabun ที่มากับเครื่องมือ (`assets/fonts`, สัญญาอนุญาต OFL) PDF จึงไม่ต้องพึ่งฟอนต์ในเครื่อง

8.2 เว็บคู่มือ (แถบบนสีแบรนด์ เมนูซ้าย "ในหน้านี้" ค้นหา ⌘K)

ส่วน	ที่อยู่
สร้างหน้า เมนู ดัชนีค้นหา	`src/build/web.ts`
สไตล์และสคริปต์ (แถบบน ค้นหาแบบพิมพ์แล้วเห็นผล ⌘K เมนูซ้าย "ในหน้านี้" ลิ้นชักบนมือถือ โหมดมืด)	`assets/web/site.css`, `assets/web/site.js`
ตั้งค่าของโครงการ	`web.brand` (ชื่อ โลโก้ สี ฟอนต์), `web.backLink`, `web.audiences`, `web.groups`

ค้นหาภาษาไทยตัดคำด้วย `Intl.Segmenter` ในเบราว์เซอร์ เว็บไม่โหลดอะไรจากโดเมนอื่นและใช้ที่อยู่แบบสัมพัทธ์ เปิดจากดิสก์ได้ เทสต์ `tests/web-theme.test.ts` รัน axe-core ทั้งโหมดสว่างและมืด แก้ธีมแล้วต้องผ่านเทสต์นี้

8.3 DOCX

ขั้นตอน: pandoc สร้างไฟล์จาก Markdown (target `pandoc`) ด้วย reference ที่สร้างจากค่าเริ่มต้นของ pandoc ตอน build (`src/build/docx.ts`) แล้วตัวประมวลผลหลังสร้างแก้ให้ถูกใน Word: ขนาดภาพ สารบัญที่มองเห็นตอนเปิด ฟอนต์ฝัง และกฎภาษาไทย

แม่แบบของโครงการ: `templates.docx` (ไฟล์ .docx ของลูกค้า) ใช้เป็น `--reference-doc` ตามที่เป็น ฟอนต์จะใส่ให้เฉพาะเมื่อกำหนด `fonts.docx` (แม่แบบของลูกค้ามักมีฟอนต์ของตัวเอง) แม่แบบสำเร็จรูปตามลูกค้าเป็นงานที่ตัดสินใจไว้ว่าจะทำทีหลัง

8.3.1 กฎ DOCX ภาษาไทย (`src/build/thai.ts`)

Word เลือกฟอนต์ทีละตัวอักษร และภาษาไทยอยู่ในช่อง "อักษรเชิงซ้อน" (complex script) ถ้าตั้งไม่ครบ Word จะใช้ฟอนต์อื่นแทน ตัวหนังสือเล็กผิดขนาด หรือตัวหนาไม่ติด กฎที่ทุก run ทุก style และค่าเริ่มต้นของเอกสารต้องมี:

กฎ	ทำไม
`w:rFonts` ascii/hAnsi = ฟอนต์ละติน, eastAsia/**cs** = ฟอนต์ไทย	ไม่มี `cs` = Word เปลี่ยนฟอนต์ไทยเอง
`w:szCs` เท่ากับ `w:sz`	ไม่มี = ตัวไทยใช้ขนาดค่าเริ่มต้น (เล็กกว่าตัวละติน)
`w:bCs` / `w:iCs` คู่กับ `w:b` / `w:i`	ไม่มี = ตัวหนา/เอียงไม่ติดกับตัวไทย
`w:lang w:bidi="th-TH"` และ `themeFontLang w:bidi="th-TH"`	Word ตัดคำและเลือกฟอนต์ตามภาษา
ฝังฟอนต์ (`.odttf` แบบ obfuscated ตาม ECMA-376)	เครื่องผู้อ่านที่ไม่มี Sarabun ยังเห็นฟอนต์ที่ถูก
`docx.thaiJustify`: `thaiDistribute` + U+200B ที่ขอบคำ	จัดชิดขอบแบบไทยโดยไม่ยืดทีละตัวอักษร (ปิดไว้เป็นค่าเริ่มต้น)

กฎใช้ซ้ำหลัง LibreOffice หรือ Word บันทึกไฟล์ทับ เพราะโปรแกรมเหล่านั้นทิ้งฟอนต์ที่ตัวเองไม่มี `manual-kit check --docx <ไฟล์>` (`src/verify-docx.ts`) ตรวจกฎทั้งหมด ลำดับ element ตาม schema และความครบของแพ็กเกจ และ build DOCX ทุกครั้งรันการตรวจนี้ให้เอง

8.3.2 เลขหน้าในสารบัญ (`src/build/finalize.ts`)

เลขหน้ามีได้หลังโปรแกรมประมวลผลคำจัดหน้าแล้วเท่านั้น `--finalize auto` ใช้ LibreOffice แบบ headless (profile ของตัวเอง ไม่แตะตัวที่ผู้ใช้เปิดอยู่) ถ้ามี ไม่มีก็ปล่อยสารบัญที่มีรายการแต่ไม่มีเลขหน้า และตั้งให้ Word ถามอัปเดตตอนเปิด **Microsoft Word ใช้เฉพาะเมื่อสั่ง `--finalize word` ตรง ๆ** ห้ามเปลี่ยนให้ auto เลือก Word (ดู บทเรียนที่ได้)

8.4 หน้าที่เกี่ยวข้อง

- เพิ่มผลลัพธ์หรือช่องทางเผยแพร่
- บทเรียนที่ได้

9 แก้และทดสอบ skill

skill สองตัวของ manual-kit คือคำสั่งที่ Claude อ่านแล้วทำตาม: **manual** บอก **ลำดับงาน** ของ `/manual` (โหมดใหม่ spec update check) ส่วน **building-product-manuals** เป็น **ที่เดียว** ของกติกาการเขียนคู่มือ การแก้ skill จึงเหมือนแก้โค้ด: ต้องมีหลักฐานว่า Claude ทำตามจริง ไม่ใช่แค่อ่านแล้วดูดี

9.1 ก่อนเริ่ม

- รู้ว่ากำลังแก้ปัญหาอะไร โดยมีตัวอย่างงานที่ Claude ทำผิด (เช่น เดาชื่อผู้อนุมัติ, ลืมปิดเซิร์ฟเวอร์ที่เปิด)
- ถามก่อนว่าปัญหานั้นเป็นกฎตรวจได้ไหม (เพิ่มกฎตรวจ) กฎที่เครื่องตรวจได้แข็งแรงกว่าข้อความใน skill
- มีโครงการทดสอบที่รันได้ในเครื่องและมีข้อมูลสมมติเท่านั้น (เช่น สำเนาของ `tests/fixtures/discover-app` หรือแอปตัวอย่างของทีม)

9.2 ขั้นตอน

1. **วางกฎไว้ที่เดียว**: กติกาและวิธีการอยู่ใน `building-product-manuals/SKILL.md` ส่วน `manual/SKILL.md` อ้างไปที่นั่น อย่าคัดลอกกฎซ้ำสองที่ เพราะวันหนึ่งจะขัดกัน รายละเอียดคำสั่งอยู่ใน `docs/reference.md` ซึ่ง `pnpm bundle` คัดลอกไปเป็น `building-product-manuals/reference.md` (แก้ที่ `docs/reference.md` เท่านั้น)
2. **วัดก่อนแก้ (baseline)**: ติดตั้ง plugin รุ่นปัจจุบันใน `CLAUDE_CONFIG_DIR` ชั่วคราว (ตั้งเครื่องสำหรับพัฒนา) เปิด Claude Code ในสำเนาโครงการทดสอบ สั่งงานเดียวกับที่เคยผิด แล้วบันทึกว่า Claude ทำอะไร (คำสั่งที่รัน ไฟล์ที่ได้ สิ่งที่ผิด)
3. **แก้ skill** ให้สั้นและเป็นคำสั่ง: บอกว่าทำอะไร **และทำไม** ระบุสิ่งที่ห้ามทำตรง ๆ เพิ่มลงหัวข้อ "Common mistakes" ถ้าเป็นความผิดที่พบบ่อย
4. **วัดหลังแก้ (with skill)**: ล้างสำเนาโครงการทดสอบ ใช้ plugin ที่แก้แล้ว (`marketplace add` ชี้ที่ repo ในเครื่อง) สั่งงานเดิมด้วยข้อความเดิม แล้วเทียบกับ baseline: ความผิดเดิมหายไหม มีความผิดใหม่ไหม
5. ทดสอบโหมดอื่นที่อาจกระทบ (`/manual spec`, `/manual update`, `/manual check`) อย่างน้อยหนึ่งรอบ
6. ลบโฟลเดอร์ config ชั่วคราวและสำเนาโครงการ ปิดทุกโปรเซสที่ตัวเองเปิด แล้ว commit skill พร้อมบันทึกผลเทียบสั้น ๆ ในข้อความ commit

9.3 ผลลัพธ์ที่ได้

skill ที่มีหลักฐานว่าทำให้ Claude ทำงานถูกขึ้น และไม่มีกฎที่ขัดกันระหว่างไฟล์ ทีมได้รับเมื่อ `claude plugin update`

9.4 ปัญหาที่พบบ่อย

อาการ	สาเหตุ	วิธีแก้

Claude ยังทำผิดเหมือนเดิม	กฎอยู่ลึกในข้อความยาว หรือไม่มีเหตุผลกำกับ	ย้ายขึ้นที่ที่ Claude อ่านก่อน บอกเหตุผล หรือเปลี่ยนเป็นกฎของ check
ทดสอบแล้ว plugin จริงของตัวเองเปลี่ยน	ไม่ได้ตั้ง `CLAUDE_CONFIG_DIR`	ตั้งทุกครั้ง ใช้โฟลเดอร์ทิ้งได้
`reference.md` ใน skill ถูกเขียนทับ	แก้ที่สำเนา	แก้ `docs/reference.md` แล้ว `pnpm bundle`
Claude รับรองหน้าเอง	กฎ "คนเท่านั้นที่รับรอง" ไม่ชัด	กฎนี้ต้องอยู่ทั้งใน skill และในข้อความของ check (มีแล้ว) ห้ามลบ

9.5 หน้าที่เกี่ยวข้อง

- กลยุทธ์การทดสอบ
- ปล่อยเวอร์ชันและกติกาการทำงาน

10 กลยุทธ์การทดสอบ

ผู้ใช้ manual-kit เห็นสองอย่าง: ผลของคำสั่ง (ข้อความ exit code) และไฟล์ที่ได้ (PDF DOCX เว็บ ภาพ) เทสต์จึงตรวจสองอย่างนั้น ไม่ตรวจไส้ในของฟังก์ชัน (หลักการ #20) ถ้าเทสต์ยังผ่านแม้ฟังก์ชันที่ import มาคืน `undefined` เทสต์นั้นไม่ได้พิสูจน์อะไร

10.1 รันอย่างไร

คำสั่ง	ทำอะไร
`pnpm test`	vitest ทุกไฟล์ใน `tests/`
`pnpm exec vitest run tests/check.test.ts`	ไฟล์เดียว
`pnpm verify`	typecheck + bundle + test + build (ต้องผ่านก่อน push)

10.2 ส่วนประกอบ

ส่วน	หน้าที่
`tests/helpers/cli.ts`	`runCli(args, cwd)` รัน `src/cli.ts` ด้วย Node แบบที่ผู้ใช้รัน (ไม่มีขั้น build) และ `runCliAsync` เมื่อเทสต์ต้องเซิร์ฟ HTTP พร้อมกัน
`tests/fixtures/project`	โครงการตัวอย่างสองภาษาที่มีคู่มือครบ ผ่านทุกกฎ (`cleanFixture()` คัดลอกไป temp ทุกเทสต์)
`tests/fixtures/next-app`, `next-pages`, `nuxt-app`, `nuxt-plain`, `discover-app`	โครงการเล็กของแต่ละเฟรมเวิร์ก สำหรับ adapter และ discover
`tests/helpers/server.ts`	เว็บเซิร์ฟเวอร์ในเครื่องสำหรับ capture
`tests/helpers/outline-mock.ts`	Outline API จำลองสำหรับ publish
`tests/helpers/pdf.ts`, `docx.ts`, `png.ts`	อ่านข้อความ PDF ด้วย `pdftotext`, แตก DOCX เป็น XML, อ่านขนาด/พิกเซลภาพ

10.3 สิ่งที่ตรวจในแต่ละแบบ

- **check**: ทำ fixture ให้ผิดหนึ่งจุด รัน `check --json` ตรวจชื่อกฎ ความรุนแรง ไฟล์ และข้อความ (เพิ่มกฎตรวจ)
- **PDF**: ข้อความไทยออกมาถูก (ไม่มีกล่องสี่เหลี่ยม) สารบัญมีเลขหน้า หัว-ท้ายกระดาษ คำบรรยายภาพ
- **DOCX**: XML ทุก run มี `cs`, `szCs`, `bCs` ตามกฎภาษาไทย ฟอนต์ฝัง แพ็กเกจครบ ลำดับ element ตาม schema

- **เว็บ**: หน้าที่สร้าง ค้นหาภาษาไทย และ axe-core ผ่านทั้งโหมดสว่างและมืด
- **capture**: ภาพมีกรอบแดงที่ตำแหน่งถูก ส่วนที่ปิดบังเป็นสีเทา action ทำงานก่อนถ่าย host นอก allowlist ถูกปฏิเสธ
- **plugin**: bundle ตรงกับซอร์ส (`is committed as built from the current source`), ตัวเรียก bash รันบน `/bin/bash` ของ macOS และหาโฟลเดอร์ข้อมูลจากโครงของ plugin cache ถูก แพ็กเกจ runtime โหลดจาก `MANUAL_KIT_HOME`

10.4 หลักการเพิ่มเทสต์

1. เขียนเทสต์ที่ไม่ผ่านก่อน แล้วค่อยแก้โค้ด (TDD) ข้อความ `it(...)` อ่านเป็นประโยคพฤติกรรม เช่น `select (by value or label), check, uncheck, hover and press on an element run before the image`
2. ใช้ fixture ที่มีอยู่ก่อนสร้างใหม่ fixture ใหม่ต้องเล็ก มีเฉพาะกรณีที่ต้องการ และเป็นข้อมูลสมมติ
3. เทสต์ที่ต้องใช้โปรแกรมภายนอก (pandoc, ffmpeg) ให้ข้ามพร้อมเหตุผลเมื่อไม่มีโปรแกรม ไม่ใช่ล้ม
4. เทสต์ต้องให้ผลเดิมทุกครั้ง: ไม่พึ่งเวลา เครือข่ายภายนอก หรือ config ของ Claude Code บนเครื่อง

10.5 หน้าที่เกี่ยวข้อง

- ตั้งเครื่องสำหรับพัฒนา
- ปล่อยเวอร์ชันและกติกาการทำงาน

11 ปล่อยเวอร์ชันและกติกาการทำงาน

ทีมติดตั้ง manual-kit จาก repo บน GitLab โดยตรง สิ่งที่อยู่บน `main` จึงคือสิ่งที่ทีมได้เมื่ออัปเดต หน้านี้บอกขั้นตอนตั้งแต่แก้โค้ดจนทีมได้ใช้ และกติกาที่ทำให้ `main` ใช้งานได้ตลอด

11.1 ก่อนเริ่ม

- งานผ่าน `pnpm verify` และทดสอบ plugin ใน `CLAUDE_CONFIG_DIR` ชั่วคราวแล้วเมื่อแก้ skill หรือตัวเรียก (ตั้งเครื่องสำหรับพัฒนา, แก้และทดสอบ skill)
- อ่าน `docs/engineering-principles.md` (สรุป 23 หลักการ เครดิต pstack ของ Lauren Tan สัญญาอนุญาต MIT)

11.2 ขั้นตอน

1. **commit ทีละเรื่อง** แต่ละ commit มีเทสต์ โค้ด bundle และเอกสารของเรื่องนั้นครบ ข้อความขึ้นต้นด้วยชนิด (`feat(capture): ...`, `fix(docx): ...`, `docs: ...`) และบอกพฤติกรรมที่เปลี่ยน
2. **บันทึกใน CHANGELOG.md** หัวข้อ `Unreleased` หนึ่งย่อหน้าต่อความสามารถ อัปเดต `docs/reference.md` ถ้าคำสั่งหรือ config เปลี่ยน และ **อัปเดตคู่มือเล่มนี้** (`docs/manual`): รัน `node dist/cli.js -m docs/manual check --allow-unsigned` check จะชี้หน้าที่ไฟล์ต้นทางเปลี่ยนและคำสั่งใหม่ที่ยังไม่มีหน้า
3. **push `main`** plugin ใช้ commit เป็นเวอร์ชัน (`claude plugin list` แสดง `Version: <commit>`) ทีมได้ของใหม่เมื่อ:

   ```
   claude plugin update manual-kit@manual-kit
   ```

   หรือเปิดอัปเดตอัตโนมัติใน `/plugin` → Marketplaces → manual-kit → Enable auto-update
4. ถ้าแพ็กเกจ runtime เปลี่ยนรุ่น (`plugins/manual-kit/runtime/package.json`) เครื่องของทีมจะติดตั้งรุ่นใหม่เองครั้งถัดไปที่เปิด Claude Code
5. **CI ของโครงการลูกค้า** ตรึง manual-kit ด้วย `MANUAL_KIT_SPEC` (commit SHA เต็ม) จึงไม่เปลี่ยนตาม `main` แจ้งทีมเมื่อมีกฎตรวจใหม่ที่ควรเลื่อน SHA (กฎใหม่อาจทำให้ merge request ที่เคยผ่านไม่ผ่าน)
6. ตัดเวอร์ชันแบบมีเลข: `package.json` ตอนนี้คือ 0.1.0 และ CHANGELOG มีหัวข้อ `0.1.0` นโยบาย tag และเลขรุ่นต่อจากนี้: (รอระบุ)

11.3 ผลลัพธ์ที่ได้

ทีมได้ความสามารถใหม่ด้วยคำสั่งเดียว `main` ผ่าน verify ทุก commit และ CHANGELOG กับคู่มือบอกสิ่งที่เปลี่ยน

11.4 กติกาการทำงานใน repo นี้

- **TDD**: เทสต์ที่ไม่ผ่านก่อน แล้วโค้ด (กลยุทธ์การทดสอบ)
- **commit เล็ก ทีละเรื่อง** ลำดับ commit อ่านแล้วเข้าใจได้ (หลักการ #19)
- **ไม่มี AI ในเครื่องมือ** งานที่ต้องใช้วิจารณญาณอยู่ใน skill (สถาปัตยกรรม)
- **ทำซ้ำได้ผลเดิม**: ไม่มี timestamp ในไฟล์ที่ commit, mapping ที่ทำให้ส่งซ้ำไม่สร้างซ้ำ (หลักการ #14)
- **ความปลอดภัย**: ถ่ายภาพเฉพาะ host ใน allowlist, ความลับจากตัวแปรสภาพแวดล้อมเท่านั้น, ไม่ใช้ Word อัตโนมัติ, ไม่ทดสอบด้วย config จริงของ Claude Code
- **บทเรียนกลายเป็นโครงสร้าง**: เจอปัญหาซ้ำ → เทสต์หรือกฎตรวจ ไม่ใช่ข้อความเตือน (หลักการ #23, บทเรียนที่ได้)

11.5 ปัญหาที่พบบ่อย

อาการ	สาเหตุ	วิธีแก้
ทีมอัปเดตแล้วได้ของเก่า	marketplace cache	`claude plugin marketplace update manual-kit` แล้ว `plugin update`
CI ของโครงการล้มหลังอัปเดต	กฎตรวจใหม่	แก้คู่มือตามที่ check บอก หรือคง SHA เดิมไว้ก่อน
`main` ใช้ไม่ได้บนเครื่องทีม	push โดยไม่ bundle	เทสต์ bundle กันไว้แล้ว: ห้ามข้าม `pnpm verify`

11.6 หน้าที่เกี่ยวข้อง

- บทเรียนที่ได้
- ดู คู่มือการใช้งาน: ติดตั้ง plugin

12 บทเรียนที่ได้

แต่ละข้อคือปัญหาที่เกิดจริงตอนสร้าง manual-kit (อ้าง commit ใน git log) พร้อมสิ่งที่เปลี่ยนในโค้ดหรือเทสต์ อ่านก่อนแก้ส่วนที่เกี่ยวข้อง เพื่อไม่ทำให้ปัญหาเดิมกลับมา

12.1 Microsoft Word ค้างระหว่างทำงานอัตโนมัติ

- **อาการ**: build DOCX ที่สั่ง Word ผ่าน AppleScript เพื่อใส่เลขหน้าในสารบัญ ค้างไม่จบ
- **สาเหตุ**: Word เปิดกล่องข้อความรอคนตอบ (เช่น ขอสิทธิ์เข้าถึงไฟล์) ซึ่งไม่มีใครเห็น และ osascript ที่ถูก kill อาจค้างตอนปิดตัว
- **สิ่งที่เปลี่ยน** (commit c805472): `--finalize auto` ใช้ LibreOffice แบบ headless เท่านั้น Word ใช้เฉพาะ `--finalize word` ที่สั่งตรง ๆ, ทุก build พิมพ์ว่าเลือกทางไหนเพราะอะไร, การเรียกภายนอกมีเวลาจำกัดและไม่รอ event ปิดตัว (`src/build/finalize.ts`) กติกา "ไม่ใช้ Word อัตโนมัติ" อยู่ใน skill และในคู่มือ

12.2 ฟอนต์ไทยใน DOCX ถูกแทนด้วยฟอนต์อื่น

- **อาการ**: เปิดใน Word แล้วตัวไทยเป็นฟอนต์อื่น ขนาดเล็ก ตัวหนาไม่ติด
- **สาเหตุ**: style บอกชื่อ Sarabun แต่ไม่ได้ฝังฟอนต์ และเครื่องส่วนใหญ่ไม่มี Sarabun อีกทั้ง run ไม่มีช่อง complex script (`cs`, `szCs`, `bCs`) ที่ Word ใช้กับภาษาไทย
- **สิ่งที่เปลี่ยน** (commit 137c5c3, ae6d335): ฝังทุกหน้าฟอนต์เป็น `.odttf` (คีย์จากไบต์ของฟอนต์ build ซ้ำได้ผลเดิม), กฎภาษาไทยครบทุก run และ style (`src/build/thai.ts`), `check --docx` ตรวจทุกครั้งที่ build ดู ธีม แม่แบบลูกค้า และกฎ DOCX ภาษาไทย

12.3 ข้อความถูก escape สองครั้ง

- **อาการ**: ป้ายที่อ้างแบบมีเครื่องหมายคำพูดขึ้นเป็น `"` บนเว็บ และ `<b>` หายใน DOCX
- **สาเหตุ**: ข้อความขั้นตอนและคำบรรยายถูก escape ทั้งตอนแทน token และตอน render
- **สิ่งที่เปลี่ยน** (commit 5993697): escape ครั้งเดียวที่จุดเดียวใน `src/render.ts` (token ที่แทนแล้วพักไว้เป็นตัวแทนระหว่าง render) เทสต์ `tests/escaping.test.ts` ตรวจ `"`, `&`, `<`, `>` ในเว็บ PDF DOCX และ WebVTT

12.4 CSS ถูกบล็อกระหว่างถ่ายภาพ

- **อาการ**: ภาพหน้าจอไม่มีสไตล์ ฟอนต์ผิด แต่ capture รายงานว่าสำเร็จ
- **สาเหตุ**: capture บล็อกทุก request นอก allowlist (กันการเปิดระบบจริง) รวมถึง CSS และฟอนต์จาก CDN
- **สิ่งที่เปลี่ยน** (commit 56f58c6): `capture.assetHosts` อนุญาตไฟล์ CSS/ฟอนต์/ภาพจาก host นั้นโดยไม่อนุญาตให้เปิดหน้าไปที่นั่น และ stylesheet ที่ถูกบล็อกทำให้ภาพนั้น **ล้ม** (ลบภาพ ไม่ลง manifest exit 1) พร้อมบอกวิธีแก้ ความผิดพลาดจึงไม่เงียบอีก

12.5 bash 3.2 บน macOS

- **อาการ**: `bin/manual-kit` พังบน macOS แต่ผ่านบน Linux
- **สาเหตุ**: macOS มากับ bash 3.2 ซึ่ง parse `case ... esac` ภายใน `$(...)` ไม่ได้
- **สิ่งที่เปลี่ยน** (commit cf4ffa4): ย้ายไปเป็นฟังก์ชัน (`plugin_root`) พร้อมคอมเมนต์เหตุผล เทสต์รันตัวเรียกด้วย `/bin/bash`

12.6 การหาโมดูลแบบ ESM ใน bundle

- **อาการ**: CLI ที่ bundle แล้วหา playwright, pagedjs และ typescript ไม่เจอ และหาไฟล์ assets ไม่เจอ
- **สาเหตุ**: ตัวโหลด ESM ของ Node ไม่สนใจ `NODE_PATH` แพ็กเกจที่ติดตั้งในโฟลเดอร์ข้อมูลของ plugin จึง import ตรงไม่ได้ และที่อยู่ของ assets ต่างกันระหว่าง repo กับ bundle
- **สิ่งที่เปลี่ยน** (commit 518ed5d): `src/deps.ts` ใช้ `createRequire` ที่ปักรากไว้ที่ `MANUAL_KIT_HOME` และโหลดเมื่อใช้ครั้งแรก (`lazyDep`) ไฟล์ที่ส่งไปกับเครื่องมือหาจากรากเดียว `src/paths.ts` ซึ่งอยู่ห่างจาก assets เท่ากันทั้งสองแบบ

12.7 คำบรรยายวิดีโอขึ้นสองชั้น

- **อาการ**: ในเว็บคู่มือ คำบรรยายของวิดีโอขึ้นซ้อนกันสองชุด
- **สาเหตุ**: คำบรรยายถูกเขียนลงบนภาพ (burned in) แล้ว แต่ track WebVTT ก็เปิดโดยค่าเริ่มต้นด้วย
- **สิ่งที่เปลี่ยน** (commit b0e2e11): track ยังมีให้ (สำหรับโปรแกรมอ่านหน้าจอและผู้ที่ต้องการ) แต่ปิดไว้จนผู้อ่านกดปุ่มคำบรรยาย

12.8 บทเรียนจากการทำคู่มือเล่มนี้

- check ของเล่มนี้จับคำโอ้อวดที่หลุดมาในประโยคธรรมดาได้ (เช่น ประโยคที่ขึ้นต้นด้วยคำว่า "ที่สุด") และจับป้ายของคำสั่ง `setup` / `doctor` ที่ adapter ยังไม่รู้จักได้ ทั้งสองแก้ก่อน build
- `init` ตั้ง `projectRoot: ..` เสมอ คู่มือที่อยู่ลึกสองชั้น (`docs/manual`) ต้องแก้เป็น `../..` เอง และ `discover` ก่อน `init` สแกนโฟลเดอร์แม่ของคู่มือแทนทั้ง repo (บันทึกเป็นปัญหาของระบบในสเปกของเล่มนี้)

12.9 หน้าที่เกี่ยวข้อง

- ธีม แม่แบบลูกค้า และกฎ DOCX ภาษาไทย
- ปล่อยเวอร์ชันและกติกาการทำงาน

ภาคผนวก

ติดต่อ

(รอระบุ) ช่องทางติดต่อผู้ดูแล manual-kit